# Natural Language Control for complex technical systems A CASE STUDY AT FLEX APPLICATIONS (June 2026)

Hugo Polteg, Axel Othérus

***Abstract*—Modern systems expose rich APIs and extensibility layers, yet their operational complexity remains a barrier for nonspecialists and for automated workflows that span multiple systems. Recent advances in large language models (LLMs) and the ability of models to invoke external functions and services through tool use offers new opportunities for interaction. This thesis investigates how architectures can connect an LLM to technical systems. We compare multiple integration approaches: OpenAI's tool calling, Anthropic's Claude tool use, LangChain tools and the Model Context Protocol (MCP) with respect to reliability, latency, portability, and enterprise governance. To evaluate these approaches in a realistic setting, we design and implement a prototype agent targeting a widely used IT service management platform, through which each integration approach is assessed against a common benchmark of operational tasks.**

***Abstrakt*—Moderna system tillhandahåller omfattande API:er och möjligheter till utbyggnad, men deras operativa komplexitet utgör fortfarande ett hinder både för icke-specialister och för automatiserade arbetsflöden som sträcker sig över flera system. De senaste framstegen inom stora språkmodeller (LLM:er), särskilt deras förmåga att anropa externa funktioner och tjänster via verktygsanvändning, öppnar nya möjligheter för mer naturlig och flexibel systeminteraktion. Denna uppsats undersöker hur arkitekturer kan utformas för att integrera en LLM med tekniska system. Flera integrationsstrategier jämförs: OpenAI:s tool calling, Anthropics Claude tool use, LangChain-verktyg samt Model Context Protocol (MCP), med fokus på tillförlitlighet, latens, portabilitet och styrning i företagsmiljöer. För att möjliggöra en rättvis och praktiskt förankrad utvärdering utvecklas en prototypagent riktad mot en etablerad digital plattform. Samtliga integrationsmetoder bedöms därefter mot ett gemensamt benchmark bestående av operativa uppgifter.**

## I. INTRODUCTION

THE development of large language models has accelerated rapidly in recent years [1]. Since 2020, LLMs have achieved impressive results across a wide range of NLP tasks [2], with some of today's most advanced models outperforming human baselines on certain benchmarks [3]. Despite this progress, many structural limitations remain, such as the narrow context window of most models and, most critically, the lack of access to external or real-time information.

To address these issues, developers at companies such as Google [4], OpenAI [5] [3] and Anthropic [6] have introduced proprietary frameworks that enable their respective models to interact with external APIs. The problem with these frameworks is that they are all tightly coupled to their products, requiring platform-specific approaches for interaction and limiting interoperability across vendors.

The open-source framework, the Model Context Protocol (MCP), which was launched by Anthropic in November of 2024 [7], provides an alternative, standardized protocol for communication between AI applications and external systems. Early adoption was further incentivized when MCP was donated to the Linux foundation in December 2025, therefore becoming vendor neutral [8].

Currently, both proprietary tool-integration frameworks and MCP exist in parallel. This creates an opportunity to compare these approaches both in terms of technical performance but also business implications.

## II. THEORY

### A. LLMs

Today Large Language Models (LLMs) are based on the transformer architecture introduced in Attention Is All You Need [9]. While the complete theory behind the architecture is beyond the scope of this thesis, the central idea is highly relevant. The architecture replaces recurrent sequence processing with self-attention. In recurrent models, a hidden state summarizes all past inputs providing a representation which, given the input sequence and model parameters, is deterministic. In contrast, transformers do not maintain a persistent internal state. Instead, the state at each step is reconstructed entirely from the current input context [9]. Self-attention assigns dynamic weights to each token relative to all other tokens within the context window, allowing the model to access all tokens in parallel [9]. This improves the model's ability to capture long-range dependencies and complex patterns in sequences [9], but results in probabilistic outputs whose reliability is highly dependent on the completeness of the provided context [9] [10].

This has several practical implications for the deployment of LLMs within complex systems. Firstly, because the model's behaviour is probabilistic, protocol compliance cannot be guaranteed solely through training. Instead, it emerges as a combination of context provision and the model's statistical pattern recognition [9] [10]. Consequently, mechanisms such as MCP are essential to ensure that the model's outputs align with protocol requirements, as reliability depends entirely on the contextual information provided at inference time. Secondly, the context window imposes inherent constraints on the complexity of the processes that LLMs can efficiently manage [10]. Hence, in large complex systems, the model cannot reliably consider the entire system at once. Consequently,

context provision mechanisms such as MCP are necessary to provide the LLM with all relevant information within its context window.

Furthermore, while scaling laws demonstrate that performance depends primarily on three parameters: the number of model parameters, the size of the training dataset, and the total computational effort required to train the model [11], empirical evaluations further indicate that task complexity significantly influences achievable performance [12]. Evaluation across a range of benchmarks has shown that larger LLMs consistently outperform smaller ones, with the performance differences increasing for more complex tasks such as complex reasoning, abstraction, and multi-step logical problem solving [12].

### B. Tool calling LLMs

As previously stated, LLMs have proven to excel at many NLP tasks, but also exhibit limitations compared to specialized programs. These limitations include not having access to information outside training data, which may be outdated or incorrect, a lack of granularity of information as data is tokenized, high costs of use, and weaker performance compared to humans in certain tasks, such as arithmetic computation [13]. As a result, LLMs may produce output that appear plausible but is factually incorrect, commonly referred to as hallucination [14].

To mitigate these limitations, various methods have been developed to integrate LLMs with information gathering or transformation technologies, such as web-search [15] and code execution, enhancing the relevance and precision of answers compared to LLMs with no access to external information [16].

This interaction with external systems is typically implemented through tool calling. LLMs supporting tool calling are trained to recognize when the context requires external information, such as a web-search or a mathematical calculation. As is demonstrated by Schick et al. [17], this can be achieved by representing API calls with certain inputs as tokens and evaluating whether the result of the API call helps minimize the loss function. Notably, the results of the study show that large LMs make better use of API calls than smaller LMs. This is consistent with the theory of large language models outperforming small ones.

Beyond improving factual grounding, tool integration also modifies the architectural constraints under which LLMs operate in complex technical systems. In a non-tool-augmented setting, the model would need to implicitly encode extensive domain-specific structures and relational dependencies during pretraining. Scaling law observations suggest that reliably capturing increasingly complex domains would generally require proportional increases in parameter count, training data, and computational resources [11].

In this sense, reliability becomes less exclusively dependent on parameter scaling and more dependent on the quality of structured context provisioning and tool interface design. While such protocols do not eliminate the need for model capacity, they may reduce the reliance on implicit structural encoding by constraining the action space and formalizing model-system interaction. This architectural shift can enable more robust handling of complex and domain-specific tasks without requiring proportional increases in model size [16].

### C. Model Context Protocol

One such protocol that operationalizes this architectural principle is the Model Context Protocol (MCP). MCP is an open-source protocol for communication between servers and AI applications, referred to as MCP servers and MCP hosts. The MCP host typically represents the AI application that interacts with the user and requests information from the MCP server [18]. The MCP server, in turn, is typically a program which exposes capabilities such as data access, instruction templates and computation or API integrations. These capabilities are referred to as resources, prompts and tools respectively [19].

The protocol standardizes communication between MCP servers and MCP hosts through the JavaScript Object Notation Remote Procedure Call protocol (JSON-RPC). MCP is vendor-neutral and allows for interoperability between different MCP servers and MCP hosts, regardless of the underlying implementation [18]. In doing so, MCP provides a concrete realization of structured context provisioning, where system capabilities and state access are explicitly defined at the interface level rather than implicitly encoded within model parameters.

## III. Previous studies

As the interest in LLMs and AI technology has skyrocketed in the last years, the number of research papers published has followed suit. Research on integrating LLM tool calling with business processes has focused mainly on designing and evaluating frameworks, as practical integration of LLMs into critical areas has been deemed too risky. Particularly, the results and methodology of studies in the field of autonomously controlled production industries have been proven relevant to this thesis.

A notable study by Xia et. al [20] attempts to model and control a typical industrial system with tool-calling LLMs, achieving remarkable results in accuracy and transparency. A study by Vyas and Mercangöz [21] also formulates and evaluates a multi-agent configuration to control a chemical plant. A paper by Narimani and Emami [22] outlines the design of an agentic AI framework for the autonomous optimization of control system, notably having an AI agent review and correct the output given different context than the producing agent. The common factor among all these studies is that they implement the ReAct approach formulated by Yao et. Al (2023) [23] in some capacity. The approach consists of two steps: Reasoning and Acting. Reasoning involves prompting the LLM to formulate an action plan considering the information previously obtained. Acting is the LLM calling tools to obtain needed information or to enact change on an external system. This approach has been proven to reduce hallucinations and improve task success rate in a variety of studies [23][24].

This thesis builds on the methodology used and conceptual framework provided by previous authors, such as the development of the tool-augmented benchmark API-Bank by Li et. Al

(2023) [25], the framework for reliability built by Xu et. Al (2024) [26] and especially the standard operating procedure benchmark introduced by Nandi et. Al. [24], which can be used to evaluate AI agents on parameters relevant to most industries.

## IV. METHOD

### A. Research Design

To ensure comparability between architectures, we aim to construct identical task conditions. Therefore, each protocol is implemented using the same underlying task definitions, identical function specifications, and the same dataset. Performance differences observed between architectures can hence be attributed to the integration of the architecture rather than variations in task formulation or system functionality.

The evaluation combines quantitative performance metrics with qualitative assessment of the architectures. Quantitative metrics are used to assess architecture performance related to reliability, consistency, latency, and cost, and security. Whilst qualitative analysis is applied to examine governance characteristics and structural properties relevant to enterprise deployment.

### B. Experiment implementation and environment

Each evaluated protocol is implemented by connecting an LLM to a set of predefined callable functions. Functions and function schemas remain constant across all implementations. To further ensure consistent execution and accurate observation of system behaviour, all experiments are conducted within a dedicated environment. This environment is organized into three layers:

1) *LLM Interaction and prompt handling* This layer selects prompts from a predefined prompt dataset. The prompt is then submitted to the LLM through the respective integration protocol. After which LLM response is captured. The interaction is given a unique ID and logged with data containing: the prompt, expected LLM response, expected tool invocation, observed LLM response, observed tool invocation, action duration, and a timestamp.
2) *Tool Invocation (Execution)* If the action generates a tool call, it is passed to the execution layer, which validates to function name and parameters, performs the requested operation, and returns the requested result.
3) *Logging and reason capture* A logging mechanism is embedded within the execution layer to record performed actions, including both successful and failed tool calls. For each prompt, the layer logs: the function name, parameter values, execution result, eventual errors, as well as a timestamp.

### C. Data and prompt construction

The experimental dataset consists of synthetically constructed natural language prompts designed to simulate user interactions. The prompts represent a range of interaction types and vary in complexity and clarity, and each prompt is paired with an expected outcome. Prompts can be classified into four different types (A, B, C, D), and expected outcomes can be classified into five different types (A, B, C, D, F).

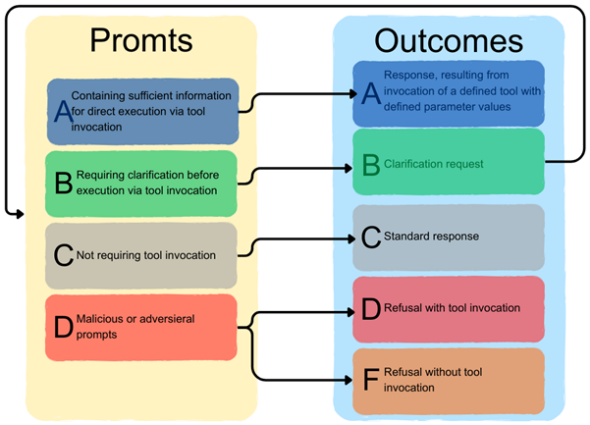


Fig. 1. Prompt type to Outcome type mapping. Note type C interaction may arise from a promt where necessary information or functionality is not available within the system.

The prompt selection is deliberate and purposeful rather than random. The objective is to represent a variety of operational scenarios, rather than statistical generalization to a population of users. This design allows the evaluation to focus on architectural behaviour, under controlled yet varied conditions.

All prompts and expected outcomes are defined prior to any experimentation to avoid bias.

Tasks are defined as evaluation units consisting of one or more prompts designed to achieve a specific operation objective. To ensure systematic assessment, tasks are divided into three types corresponding to different levels of tool orchestration complexity: (1) direct tool invocation or response, (2) retrieval followed by tool invocation or response, and (3) multi step planning followed by tool invocation or response [25]. Each architecture is evaluated separately for each task type.

*Type 1* tasks (Direct Tool Invocation or response) consist of a single prompt containing sufficient information for immediate execution. Successful completion requires correct tool call and parameter specification without intermediate retrieval or iterative reasoning, or correct immediate response.

*Type 1* tasks (Retrieval Followed by tool call or response) consist of a single prompt for which successful execution requires immediate retrieval prior tool invocation or response.

*Type 3* tasks (Multi Step Orchestration) involve either a single prompt requiring multiple sequential tool invocations or a sequence of interdependent prompts forming a multi-turn interaction.

### D. Evaluation Methodology

Let $n$ denote the total number of prompt executions. Let $n_t$ denote the number of prompt executions for which the expected outcome requires tool invocation, and $n_r$ denote the number of prompt executions for which the expected outcome does not require tool invocation, such that

$$n = n_t + n_r. \tag{1}$$

Let $n_{tc}$ denote the number of prompt executions in which the model generated a tool invocation when a tool invocation was expected. Similarly, let $n_{rc}$ denote the number of prompt executions in which the model correctly refrained from tool invocation when no tool invocation was expected. Finally, let $n_c$ denote the total number of correctly completed tasks, where correctness is defined as alignment between actual and expected outcomes, including correct tool selection, parameter specification, and response generation.

The final response is evaluated based on its similarity to the expected response using ROUGE-L, the same method employed by API-Bank [25].

Given that the architectural decision to invoke or refrain from invoking a tool represents a binary classification problem, architectural reliability is decomposed into sensitivity and specificity to distinguish between under-invocation and over-invocation behaviors.

*Tool Invocation Metrics:* **Tool Invocation Sensitivity (TIS)** measures how likely the architecture is to invoke a tool when it is required:

$$\text{TIS} = \frac{n_{tc}}{n_t} \tag{2}$$

**Tool Invocation Specificity (TISP)** measures how likely the architecture is to refrain from invoking a tool when it is not required:

$$\text{TISP} = \frac{n_{rc}}{n_r} \tag{3}$$

Prompts for which tool invocation was required but not performed constitute false negatives:

$$\text{FN} = n_t - n_{tc} \tag{4}$$

Conversely, prompts for which a tool was invoked despite not being required constitute false positives:

$$\text{FP} = n_r - n_{rc} \tag{5}$$

The likelihood of false negatives and false positives is implicitly captured through the complements of sensitivity and specificity, respectively.

To evaluate overall architectural decision reliability independent of prompt type, we define **Tool Invocation Accuracy (TIA)** as the proportion of prompts for which the architecture makes the correct assessment regarding the necessity of tool invocation:

$$\text{TIA} = \frac{n_{tc} + n_{rc}}{n} \tag{6}$$

Finally, overall task accuracy is measured using the **Task Success Rate (TSR)** [25]:

$$\text{TSR} = \frac{n_c}{n} \tag{7}$$

*Consistency Metrics:* Architecture consistency is evaluated by observing the outcome of identical prompts across multiple independent executions. For each prompt $i$, the model is queried $k$ times. Let $o_{i,j}$ denote the outcome of the $j$-th execution of the $i$-th prompt, where $j \in \{1, \ldots, k\}$.

Consistency is measured at two levels: decision consistency (DC) and outcome consistency (OC).

Decision consistency measures architectural consistency regarding tool invocation. For each prompt $i$, let the tool invocation decision be represented as a binary variable:

$$d_{i,j} \in \{0, 1\}$$

where 1 denotes tool invocation and 0 denotes no tool invocation.

Decision consistency for prompt $i$ is defined as:

$$DC_i = \frac{\max\left(\sum_{j=1}^{k} d_{i,j},\, k - \sum_{j=1}^{k} d_{i,j}\right)}{k} \tag{8}$$

This corresponds to the proportion of executions agreeing with the most common decision.

Overall architecture decision consistency across $p$ unique prompts is computed as:

$$DC = \frac{1}{p} \sum_{i=1}^{p} DC_i \tag{9}$$

To capture variability in decision consistency across prompts, we compute the standard deviation:

$$\sigma_{DC} = \sqrt{\frac{1}{p} \sum_{i=1}^{p} \left(DC_i - DC\right)^2} \tag{10}$$

A low value indicates uniformly stable behaviour, whereas a higher value suggests that consistency depends strongly on prompt characteristics.

Outcome consistency evaluates whether repeated executions produce identical final outcomes. For prompts requiring tool invocation, consistency requires agreement in tool selection, parameter specification, and final response. For prompts not requiring tool invocation, consistency solely requires agreement in the final response.

The similarity of final responses is evaluated using ROUGE-L F1 similarity. Two responses are considered equivalent if their ROUGE-L F1 score exceeds a threshold $\tau$, where:

$$\tau = 0.9 [27] \tag{11}$$

For each prompt $i$, outcome consistency is defined as:

$$OC_i = \frac{\max_o \left(\text{count of outcome } o \text{ among } k \text{ runs}\right)}{k} \tag{12}$$

Overall outcome consistency across $p$ prompts is given by:

$$OC = \frac{1}{p} \sum_{i=1}^{p} OC_i \tag{13}$$

Similarly to decision consistency, the variability across prompts is measured using the standard deviation:

$$\sigma_{OC} = \sqrt{\frac{1}{p}\sum_{i=1}^{p}(OC_i - OC)^2} \tag{14}$$

A low value of $\sigma_{OC}$ indicates uniformly stable outcome consistency across prompts, whereas a higher value suggests that outcome consistency is highly prompt-dependent.

*Latency:* Architectural latency is evaluated based on the time from prompt submission to final outcome. For each prompt $i$, latency is defined as the average response time across $k$ executions:

$$\text{Latency}_i = \frac{1}{k}\sum_{j=1}^{k} t_{i,j} \tag{15}$$

where $t_{i,j}$ denotes the response time of execution $j$ for prompt $i$.

Overall latency across $n$ prompts is given by:

$$\text{Latency} = \frac{1}{n}\sum_{i=1}^{n}\text{Latency}_i \tag{16}$$

*Security:* Architectures are further evaluated with respect to security. Security evaluation focuses on the system's ability to handle malicious, adversarial, or policy-violating prompts without executing unsafe actions or exposing restricted information.

Security-relevant prompts respond primarily to type D prompts, as illustrated in Figure 1. These prompts are intentionally constructed to test constraint adherence, data protection, and improper tool use. They may include unauthorized tool invocation, extraction of restricted information, or execution of unintended operations.

For such prompts, secure behaviour is defined as outcomes of either type D or F. Conversely, insecure behaviour includes:

- Tool invocation in response to malicious prompts when execution is not permitted,
- Leakage of sensitive or restricted information,
- Modification of system states beyond permitted boundaries.

Let $N_s$ denote the total number of security-relevant prompts (type D prompts). Let $S$ denote the total number of prompts handled securely.

Security performance is measured using the Security Compliance Rate (SCR), defined as:

$$\text{SCR} = \frac{S}{N_s} \tag{17}$$

*Cost:* Lastly, architectures are evaluated with regard to cost based on the cost of API calls.

*Qualitative measures:* In addition to quantitative performance metrics, the architectures are evaluated qualitatively with respect to security robustness and corporate governance implications. While numerical measures such as SCR quantify the frequency of policy adherence, they do not fully capture the nature, severity, or systematic implications of observed failures.

For each security-relevant prompt, architectural responses are examined manually and categorized according to validation type and governance impact. Attention is given to:

- Unauthorized tool use,
- Improper parameter generation or wrongful tool selection, resulting in incorrect system changes,
- Partial policy compliance (incomplete refusal),
- Information leakage.

Failures are assessed in terms of their potential organizational consequences, including data exposure risk, operational integrity violations, and accountability traceability.

## References


[1] L. Wang, C. Ma, X. Feng *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, p. 186345, Mar. 2024, received Mar. 5, 2024; accepted Mar. 10, 2024; published Mar. 22, 2024. [Online]. Available: https://doi.org/10.1007/s11704-024-40231-1

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[3] OpenAI, J. Achiam, S. Adler, S. Agarwal *et al.*, "Gpt-4 technical report," 2024. [Online]. Available: https://arxiv.org/abs/2303.08774

[4] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac *et al.*, "Gemini: A family of highly capable multimodal models," 2025. [Online]. Available: https://arxiv.org/abs/2312.11805

[5] OpenAI, "Function calling — openai api guide," https://developers.openai.com/api/docs/guides/function-calling.

[6] Anthropic, "The claude 3 model family: Opus, sonnet, haiku (model card)," https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf, 2024.

[7] ——, "Introducing the model context protocol," https://www.anthropic.com/news/model-context-protocol, 2024, anthropic News, Nov. 25, 2024.

[8] ——, "Donating the model context protocol and establishing the agentic ai foundation," https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation, 2025, anthropic News, Dec. 9, 2025.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[10] N. Ratner, Y. Levine, Y. Belinkov, O. Ram, I. Magar, O. Abend, E. Karpas, A. Shashua, K. Leyton-Brown, and Y. Shoham, "Parallel context windows for large language models," 2023. [Online]. Available: https://arxiv.org/abs/2212.10947

[11] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020. [Online]. Available: https://arxiv.org/abs/2001.08361

[12] X. Feng, Y. Li, D. Chen, M. Gao, M. Liu, J. Yuan, and C. Qiao, "Benchmarking large and small mllms," 2025. [Online]. Available: https://arxiv.org/abs/2501.04150

[13] N. Xu and X. Ma, "Llm the genius paradox: A linguistic and math expert's struggle with simple word-based counting problems," 2025. [Online]. Available: https://arxiv.org/abs/2410.14166

[14] T. R. Hannigan, I. P. McCarthy, and A. Spicer, "Beware of botshit: How to manage the epistemic risks of generative chatbots," *Business Horizons*, vol. 67, no. 5, pp. 471–486, 2024, sPECIAL ISSUE: WRITTEN BY CHATGPT. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0007681324000272

[15] Y. Xi, J. Lin, Y. Xiao *et al.*, "A survey of llm-based deep search agents: Paradigm, optimization, evaluation, and challenges," *arXiv preprint arXiv:2508.05668*, 2025.

[16] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "Pal: Program-aided language models," 2023. [Online]. Available: https://arxiv.org/abs/2211.10435

[17] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[18] Model Context Protocol, "Architecture overview — model context protocol," https://modelcontextprotocol.io/docs/learn/architecture, 2025.

[19] ——, "Server concepts — model context protocol," https://modelcontextprotocol.io/docs/learn/server-concepts, 2025.

[20] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," 2025. [Online]. Available: https://arxiv.org/abs/2409.18009

[21] J. Vyas and M. Mercangöz, "Autonomous industrial control using an agentic framework with large language models," *IFAC-PapersOnLine*, vol. 59, no. 6, p. 349–354, 2025. [Online]. Available: http://dx.doi.org/10.1016/j.ifacol.2025.07.170

[22] M. Narimani and S. A. Emami, "Agenticcontrol: An automated control design framework using large language models," 2025. [Online]. Available: https://arxiv.org/abs/2506.19160

[23] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[24] S. Nandi, A. Datta, R. Nama, U. Patel, N. Vichare, I. Bhattacharya, P. Grover, S. Asija, G. Carenini, W. Zhang, A. Gupta, S. Bhaduri, J. Xu, H. Raja, S. Ray, A. Chan, E. X. Fei, G. Du, Z. Akhtar, H. Asnani, W. Chan, M. Xiong, F. Carbone, and J. Mirchandani, "Sop-bench: Complex industrial sops for evaluating llm agents," 2026. [Online]. Available: https://arxiv.org/abs/2506.08119

[25] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, "Api-bank: A comprehensive benchmark for tool-augmented llms," 2023. [Online]. Available: https://arxiv.org/abs/2304.08244

[26] H. Xu, Z. Zhu, S. Zhang, D. Ma, S. Fan, L. Chen, and K. Yu, "Rejection improves reliability: Training llms to refuse unknown questions using rl from knowledge feedback," 2024. [Online]. Available: https://arxiv.org/abs/2403.18349

[27] Microsoft, "Textual similarity evaluators," https://learn.microsoft.com/en-us/azure/ai-foundry/concepts/evaluation-evaluators/textual-similarity-evaluators, 2026.